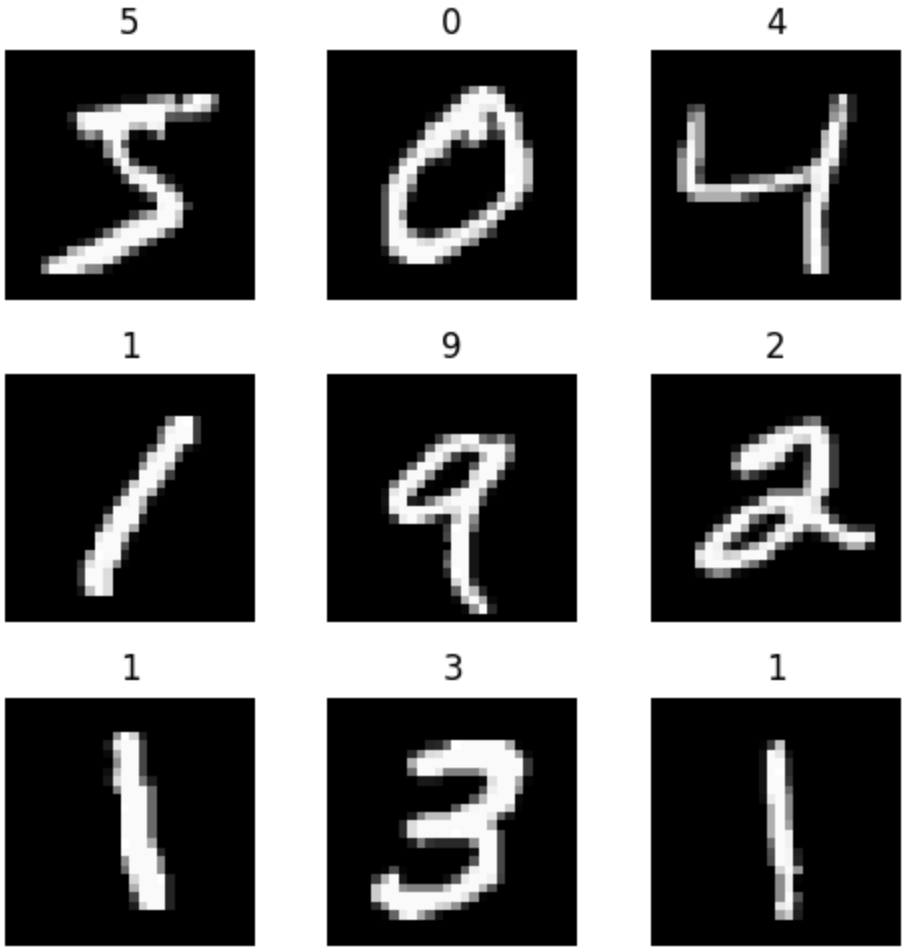
```
import sys, tensorflow as tf
print("Python:", sys.version)
print("TensorFlow:", tf.__version__)
```

```
Python: 3.12.12 (main, Oct 10 2025, 08:52:57) [GCC 11.4.0]
TensorFlow: 2.19.0
```

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist

# Load MNIST
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize (0..255 -> 0..1)
x_train = x_train / 255.0
x_test  = x_test  / 255.0

print("x_train shape:", x_train.shape, "y_train shape:", y_train.shape)
print("x_test shape:", x_test.shape, "y_test shape:", y_test.shape)

# Show 3x3 grid
plt.figure(figsize=(5,5))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(str(y_train[i]))
    plt.axis("off")
plt.tight_layout()
plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-dataset
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
x_train shape: (60000, 28, 28) y_train shape: (60000,)
x_test shape: (10000, 28, 28) y_test shape: (10000,)
```




```
model = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=(28,28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    x_train, y_train,
    epochs=5,
    validation_data=(x_test, y_test)
)

model.save("mnist_model.h5")
```

```
print("Saved: mnist_model.h5")
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37
  super().__init__(**kwargs)
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.8761 - loss: 0.4391 - v
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9662 - loss: 0.1185 - v
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9760 - loss: 0.0787 -
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9826 - loss: 0.0568 - v
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 17s 9ms/step - accuracy: 0.9868 - loss: 0.0444 -
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ke
Saved: mnist_model.h5
```

```
model = tf.keras.models.load_model("mnist_model.h5")

converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

with open("mnist_model.tflite", "wb") as f:
    f.write(tflite_model)

print("Saved: mnist_model.tflite")
print("TFLite size (bytes):", len(tflite_model))
```

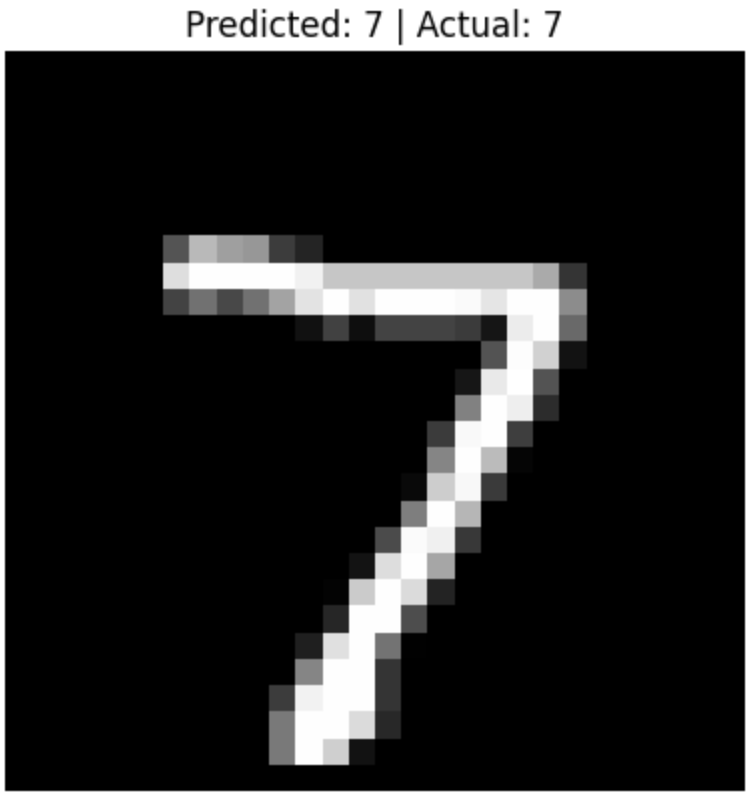
```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be
Saved artifact at '/tmp/tmp_0i28quu'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 28, 28), dtype=tf.float32, n
Output Type:
  TensorSpec(shape=(None, 10), dtype=tf.float32, name=None)
Captures:
  133441973808656: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133441973812304: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133441932417104: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133441932416720: TensorSpec(shape=(), dtype=tf.resource, name=None)
Saved: mnist_model.tflite
TFLite size (bytes): 409028
```

```
interpreter = tf.lite.Interpreter(model_path="mnist_model.tflite")
interpreter.allocate_tensors()

input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
```

```
print("Input Details:", input_details)
print("Output Details:", output_details)
```

```
Input Details: [{'name': 'serving_default_input_layer:0', 'index': 0, 'shape': a
Output Details: [{'name': 'StatefulPartitionedCall_1:0', 'index': 9, 'shape': ar
/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/interpreter.py:45
    TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
    See the [migration guide](https://ai.google.dev/edge/litert/migration)
    for details.

  warnings.warn(_INTERPRETER_DELETION_WARNING)
```

```
# Pick a test image
idx = 0
test_image = x_test[idx].astype(np.float32)
test_image = np.expand_dims(test_image, axis=0)  # (1,28,28)

# Set input tensor
interpreter.set_tensor(input_details[0]['index'], test_image)

# Run inference
interpreter.invoke()

# Get output
output_data = interpreter.get_tensor(output_details[0]['index'])
predicted_label = int(np.argmax(output_data))

# Display
plt.imshow(x_test[idx], cmap="gray")
plt.title(f"Predicted: {predicted_label} | Actual: {y_test[idx]}")
plt.axis("off")
plt.show()

print("Raw probabilities (first row):", output_data[0])
```




```
Raw probabilities (first row): [1.3568133e-08 2.2697918e-08 9.5619919e-07 4.9351
 7.3515587e-08 4.8281826e-11 9.9994421e-01 3.7769969e-06 1.6267450e-06]
```

```
from google.colab import files
files.download("mnist_model.h5")
files.download("mnist_model.tflite")
```

L02 – Deploying an AI Model on a Simulated Edge Device

Chosen Approach: Option B – Practical

Student: Oscar Cortez
Course: ITAI 3377
Date: January 26, 2026

Objective

Train a simple neural network on the MNIST dataset using TensorFlow/Keras, convert the trained model to TensorFlow Lite (TFLite), and run inference using the TFLite Interpreter

to simulate deployment on an edge device.

Part 1: Setting Up the Development Environment

In this section, I verify that Python and TensorFlow are available in the runtime. This ensures the environment can train a model and use TensorFlow Lite for conversion and inference. If TensorFlow is missing, it can be installed using `pip`.

Part 2: Loading and Preprocessing the MNIST Dataset

I load the MNIST dataset using Keras, which provides training and test splits.
To preprocess the data for training, I normalize the pixel values from **0–255** to **0–1** by dividing by 255.0. This improves training stability and helps the optimizer converge faster. I also visualize a sample grid of images to confirm the dataset loaded correctly.

Part 2: Designing and Training the Neural Network

Model Architecture (Keras)

- **Input:** 28×28 grayscale image
- **Flatten layer:** converts 2D image into a 1D vector
- **Dense (128, ReLU):** hidden layer to learn features
- **Dense (10, Softmax):** output probabilities for digits 0–9

Training

The model is compiled with:

- **Optimizer:** Adam
- **Loss:** Sparse Categorical Crossentropy
- **Metric:** Accuracy

I train for **5 epochs** and then save the trained model to disk as `mnist_model.h5`.

Part 3: Converting and Saving the Model (TensorFlow Lite)

Why convert to TensorFlow Lite?

TensorFlow Lite is designed for **efficient inference on resource-constrained devices** (edge/IoT). Converting the model produces a smaller, optimized format (`.tflite`) that can be run using the TFLite Interpreter.

Conversion Process

I load the saved Keras model (`mnist_model.h5`), convert it using `tf.lite.TFLiteConverter`, and write the converted bytes to `mnist_model.tflite`.

Part 4: Simulated Deployment Using the TensorFlow Lite Interpreter

To simulate an edge deployment, I load `mnist_model.tflite` using the **TensorFlow Lite Interpreter**.
After calling `allocate_tensors()`, I retrieve the model's **input and output tensor details** to confirm the expected shapes and data types before running inference.

Part 4: Running Inference (Testing on Sample Input)

I select a sample test image from the MNIST test set and format it to match the model's expected input:

- Convert to `float32`
- Add a batch dimension (shape becomes **(1, 28, 28)**)

Then I:

1. Set the input tensor
2. Run inference with `interpreter.invoke()`
3. Read the output tensor (probabilities)
4. Use `argmax` to select the predicted digit

Finally, I display the image and compare **predicted vs actual** labels to confirm the deployment pipeline works end-to-end.

Reflective Journal (1–2 pages)

Challenges Faced

One challenge was making sure the data format matched what TensorFlow Lite expects during inference. Even small mismatches—such as missing a batch dimension or using the wrong data type—can cause incorrect results or errors. Another challenge was keeping the workflow organized so the training model, conversion step, and interpreter testing all connected cleanly without losing track of which file was being used (`.h5` vs `.tflite`). Lastly, exporting everything into a single PDF can sometimes be tricky if notebook output formatting is inconsistent, so I had to pay attention to readability and organization.

Learning Outcomes

This lab helped me understand the complete pipeline from model training to edge-style deployment. I learned how a standard Keras model can be trained quickly on MNIST, saved, and then converted into a TensorFlow Lite model for more efficient inference. I also learned how the TensorFlow Lite Interpreter works at a lower level than Keras: you have to explicitly allocate tensors, set input data into the correct tensor index, invoke the interpreter, and then read output tensors. That process made the “deployment” phase feel more concrete and closer to how inference happens on real devices.

Application of Knowledge (Real-World / Edge & IIoT)

In real-world edge and IIoT environments, sending raw sensor or image data to the cloud can create latency, bandwidth, and privacy issues. TensorFlow Lite supports running